Lattice Scan: three-page summary

Lattice Scan: three-page summary

Cantor8 "Build on Canton" hackathon, London, 29 August 2026. Track A1 (build a scanner).

1. The problem

Canton is a blockchain built for banks. On Ethereum every node stores every transaction and anyone can read any wallet's history. Canton keeps the useful parts (one shared truth, no double spending, final settlement) and drops the part banks cannot accept: public visibility. A transaction is delivered only to the parties named on it. The service that orders transactions, the synchronizer, cannot read them. Think of private rooms with a doorman who sorts the mail but cannot open it.

The consequence is that Canton has no block explorer and cannot have one. There is no machine anywhere that knows the whole ledger. If a validator operator, a wallet, or a compliance team wants balances, history, or an activity feed, someone has to build an index of what that node is entitled to see. That index is what we are building.

2. What we got wrong, and what fixed it

Our early scanner returned one contract per party and we concluded we had hit a privacy wall: you can only see contracts you are a stakeholder of, you cannot add yourself as an observer after the fact, and nothing lets an outsider watch a wallet. All of that is true. It was also irrelevant, because the toolkit was asking the ledger a narrow question. It filtered the active contract set for the `Holding` interface, which means "token balances only". Everything else the party could see was excluded by the question, not by permissions.

The organiser's fix was two words: use the Active Contract Set with a wildcard filter. We verified it live today:

- The hackathon token belongs to the validator's backend user and carries `CanReadAsAnyParty`. The validator is a stakeholder on every transaction its hosted parties take part in, so the token can read all of them.
- One WebSocket request returned the full snapshot: 109,779 active contracts, 178 MB, in 11.9 seconds, spanning 145 contract types and 26,020 named parties, of which 20,596 are hosted on this validator.
- The plain HTTP endpoints stop at 200 elements with a 413 error. The WebSocket endpoints have no such limit. That decides the architecture.
- The Canton Coin balance we computed from the snapshot for the party `bob` (144,628.2489548600 CC) matched the network's public Scan API to ten decimal places. That gives us an independent correctness check the judges can run themselves.

Privacy did not change. We see only what parties hosted on this validator are stakeholders of. Contracts between parties on other validators never reach this node.

3. What we are building

Lattice Scan is a single Node.js program with no dependencies to install. It keeps its own copy of the ledger in one SQLite file and answers questions from that copy.

How it works, in six steps:

1. Get a badge. Ask the organiser's login server for a token. Tokens last 15 minutes, so the program renews them on its own.
2. Photograph everything. Ask the node for its full active contract set at a fixed position (the offset) and stream it into the database.

3. Write it in the notebook. Every contract is stored with its type, its data, who signed it and who may see it. Contracts that represent money (Canton Coin and Cantor8's own tokens) are also summarised into a holdings table.
4. Watch the door. From that same offset, subscribe to the live update stream. Each update's creates and archives are applied to the database together with the new offset in one all-or-nothing write. If the process is killed at any instant, it restarts from the last committed offset with nothing lost and nothing counted twice.
5. Answer questions. A JSON API serves health (offset, lag in seconds, counts), balances per party and token, holdings, transfer history, and any contract by id. A one-page dashboard reads it.
6. Re-check the notebook against reality. On demand, the program re-downloads the ledger's active set at a pinned offset and diffs it against its own database. The expected answer is zero differences, after bootstrap, after an hour of tailing, and after a forced restart. Anything that does differ is listed by contract id, which is the honest answer.

Stack: Node 24, `node:sqlite`, the browser-standard WebSocket built into Node, plain HTTP. Nothing to install, one file to back up, runs on a laptop.

4. The numbers we bring to the judges

Question the judges ask	Our answer
Does it measure the thing?	109,779 contracts indexed at bootstrap; lag in seconds shown live (the organisers' own scanner reports about 2.7 s); balances for five parties checked against the public Scan oracle
Does it survive an attack?	kill -9 during streaming, restart re-reads 0 contracts and resumes exactly, then a fresh ledger snapshot diffed against our copy shows zero differences; token expiry handled by recycling the connection every 10 minutes; duplicate deliveries are no-ops by database key
Does it work outside the demo?	node URL, login server and party scope are configuration; falls back to per-party filters if a token lacks read-any-party rights
Is the honesty good?	the boundary is printed on the dashboard: one validator's view only; history before the node's pruning point (offset 2,907,915) is not reconstructable; unclassified activity is shown as unclassified
Would this ship?	zero dependencies, health endpoint, write-ahead-logged SQLite, README with the measured numbers

5. What we are not claiming

- Not a network explorer. Other validators' private activity is invisible by design.
- Not complete history. The node has pruned everything before offset 2,907,915; the snapshot carries every active contract, the stream carries everything from now on.
- Canton Coin balances are reported as the sum of each note's face value, the same figure the Scan API and Cantor8's scanner report. The value after holding fees is shown separately.
- Transfer classification uses only create and archive events. When the counterparty is on another validator, we say the counterparty is unknown rather than guessing.

6. Where we are and what is next

Done today: the live network is characterised (token rights, snapshot size and speed, stream shapes, the 413 limit, pruning point, the Scan oracle), every contract type on the node is catalogued with its field names, and the build plan is written down to the database schema, the API and an hour-by-hour timeline.

Next, in order: bootstrap into SQLite; live tail with atomic checkpoints; health and balance endpoints; kill test; history and classification; self-check against a fresh snapshot; measurement script; dashboard if time remains.

Two things we need from the organisers in the first minutes: a party of our own with some Canton Coin (the node is quiet on a Saturday, so we generate our own transfers for the live demo), and confirmation that the day's token keeps read-any-party rights (if not, we switch to per-party filters and lose nothing else).

7. Sixty-second pitch

Canton has no block explorer, and it cannot have one. The network is built out of private rooms, and the doorman who orders the mail cannot read it. That is why banks are on it, and it means every validator has the same problem: it cannot answer basic questions about its own data.

Lattice Scan connects to one node, takes a complete snapshot of everything that node is entitled to see, and follows the ledger forward second by second. Today that is 109,779 contracts in under twelve seconds. Our Canton Coin balances match the network's public source to ten decimal places. Kill the process mid-flight and it resumes from its exact position. Re-download the ledger and diff it against our copy: zero differences.

It is not the network. It is one node's honest view of itself. Every validator needs one.